



LOVELY
PROFESSIONAL
UNIVERSITY

Transforming Education Transforming India

Course Code: CSC 401
(SECURING IT INFRASTRUCTURE)

CA 02
Submitted By: -

Sr. No.	Registration No	Name	Roll No
1	12305250	Mohak Grover	68

Lovely Professional University Jalandhar, Punjab,
India.

School of AI and Emerging Technologies

Problem Statement

IDS/IPS systems are generating excessive alerts. You must tune rules to improve accuracy and reduce false positives.

Solution

1. Introduction

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential cybersecurity tools used to monitor network traffic and detect malicious activities. Modern organizations rely on IDS solutions such as Suricata to identify threats like unauthorized access, scanning attacks, and suspicious network behavior.

However, IDS systems often generate a very large number of alerts, many of which are false positives or duplicate notifications. Excessive alerts create alert fatigue, increase analyst workload, and make it difficult to identify real attacks. Research studies explain that false positives are one of the biggest challenges in IDS environments.

Recent IDS research focuses on reducing false positives using techniques such as:

- ♦ Alert correlation
- ♦ Thresholding
- ♦ Data mining
- ♦ Machine learning

- AI-based analysis

In this project, Suricata IDS was configured on Kali Linux and custom rules were created to detect ICMP traffic, HTTP requests, Nmap scans, SSH attempts, DNS queries, FTP traffic, and MySQL connections. During testing, excessive alerts were generated for repeated network activities.

To reduce alert flooding and improve detection accuracy, thresholding and suppression techniques were implemented using Suricata's `threshold.config` file. These tuning methods helped minimize duplicate alerts while maintaining effective intrusion detection.

The project demonstrates how IDS tuning can improve monitoring efficiency, reduce false positives, and simplify security analysis in real-world cybersecurity environments.

2. Problem Statement

Intrusion Detection Systems (IDS) continuously monitor network traffic to detect suspicious activities and cyber attacks. However, default IDS configurations often generate excessive alerts for normal network behavior, resulting in a high number of false positives and duplicate alerts.

During practical implementation using Suricata IDS, repeated alerts were generated for:

- ICMP ping traffic
- HTTP requests
- SYN scan packets
- Port scanning activities

This excessive alert generation created problems such as:

- Alert fatigue
- Increased analyst workload
- Difficulty identifying real threats
- High log storage consumption

- Reduced monitoring efficiency

Research papers also highlight that IDS systems can generate thousands of unnecessary alerts daily, making manual analysis difficult for administrators.

Therefore, the main problem addressed in this project is reducing excessive IDS alerts and false positives while maintaining accurate threat detection using Suricata IDS tuning techniques such as thresholding and suppression.

3. Objectives

Explanation

The main objective of this project is to improve the efficiency of Intrusion Detection Systems (IDS) by reducing excessive alerts and false positives generated by Suricata IDS.

The project focuses on configuring and tuning Suricata rules to improve detection accuracy while minimizing unnecessary alerts. Different types of network traffic were generated and analyzed to observe IDS behavior before and after tuning.

The major objectives of this project are:

- To install and configure Suricata IDS
- To create custom detection rules
- To generate network traffic for testing
- To detect suspicious activities such as scans and connection attempts
- To analyze excessive alert generation
- To implement thresholding and suppression techniques
- To reduce false positives and duplicate alerts
- To improve IDS monitoring efficiency and alert accuracy

The project also aims to demonstrate practical IDS tuning techniques discussed in cybersecurity research papers related to false positive reduction and alert optimization.

4. Literature Review

Explanation

Several researchers have worked on reducing false positives and improving the efficiency of Intrusion Detection Systems (IDS). One of the major issues discussed in IDS research is alert flooding, where IDS systems generate a huge number of alerts, many of which are false positives.

The research paper “Reducing False Positives in Intrusion Detection Systems” explains that IDS systems generate thousands of alerts daily, making analysis difficult for administrators. The paper proposes statistical filtering techniques and alert analysis methods to reduce false positives by identifying repetitive alert patterns.

Another research paper titled “False Positives Reduction in Intrusion Detection Systems Using Alert Correlation and Data Mining Techniques” discusses how alert correlation and clustering methods can help distinguish real attacks from duplicate or unnecessary alerts.

The paper “Advancements in Intrusion Detection Systems” highlights modern IDS approaches such as:

- Hybrid IDS
- Machine learning-based IDS
- AI-powered detection
- Behavioral analysis
- Cloud-based IDS systems

These technologies improve threat detection accuracy and help reduce false positives.

Another recent study, “Leveraging Large Language Models for Reducing False Positives and Prioritizing Alerts in IDS,” explains how AI and Large Language

Models LLMs can classify alerts more accurately and reduce analyst workload in Security Operations Centers SOC .

The concepts discussed in these research papers were practically implemented in this project using Suricata IDS tuning techniques such as thresholding, suppression, and custom rule optimization.

5. Tools and Technologies Used

This project was implemented using Suricata IDS on Kali Linux to monitor and analyze network traffic. Various tools were used for traffic generation, rule configuration, and alert analysis.

Tool/Technology	Purpose
Suricata	Intrusion Detection System
Kali Linux	Operating System
Nmap	Port scanning and traffic generation
Curl	HTTP traffic generation
Ping	ICMP traffic testing
Nano Editor	Editing rule/configuration files
VMware	Virtualized testing environment

These tools were used to configure the IDS environment, create custom rules, generate testing traffic, and analyze alert behavior before and after IDS tuning.

6. System Architecture

The system architecture consists of traffic generation tools, Suricata IDS, custom detection rules, and alert tuning mechanisms. Network traffic generated using tools like Ping, Curl, and Nmap is analyzed by Suricata in real time. The IDS then generates alerts based on custom rules configured in the `local.rules` file.

To reduce excessive alerts and false positives, thresholding and suppression techniques were applied using the `threshold.config` file. This improved the overall accuracy and efficiency of alert monitoring.

Traffic Generation → Suricata IDS → Alert Generation

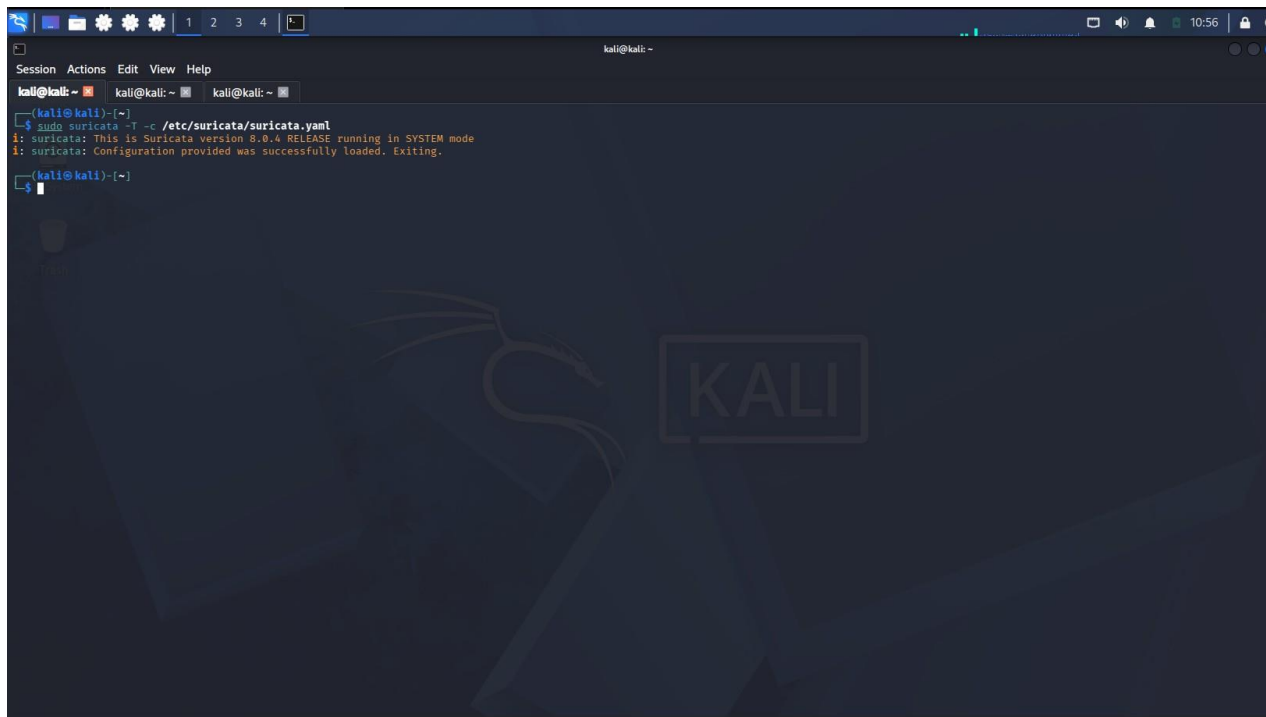


Rule Tuning & Thresholding



Reduced False Positives

Configuration files and threshold settings successfully stored and applied in Suricata IDS used for validating the configurations



```
kali@kali: ~  
└─$ sudo suricata -T -c /etc/suricata/suricata.yaml  
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode  
i: suricata: Configuration provided was successfully loaded. Exiting.  
└─$
```

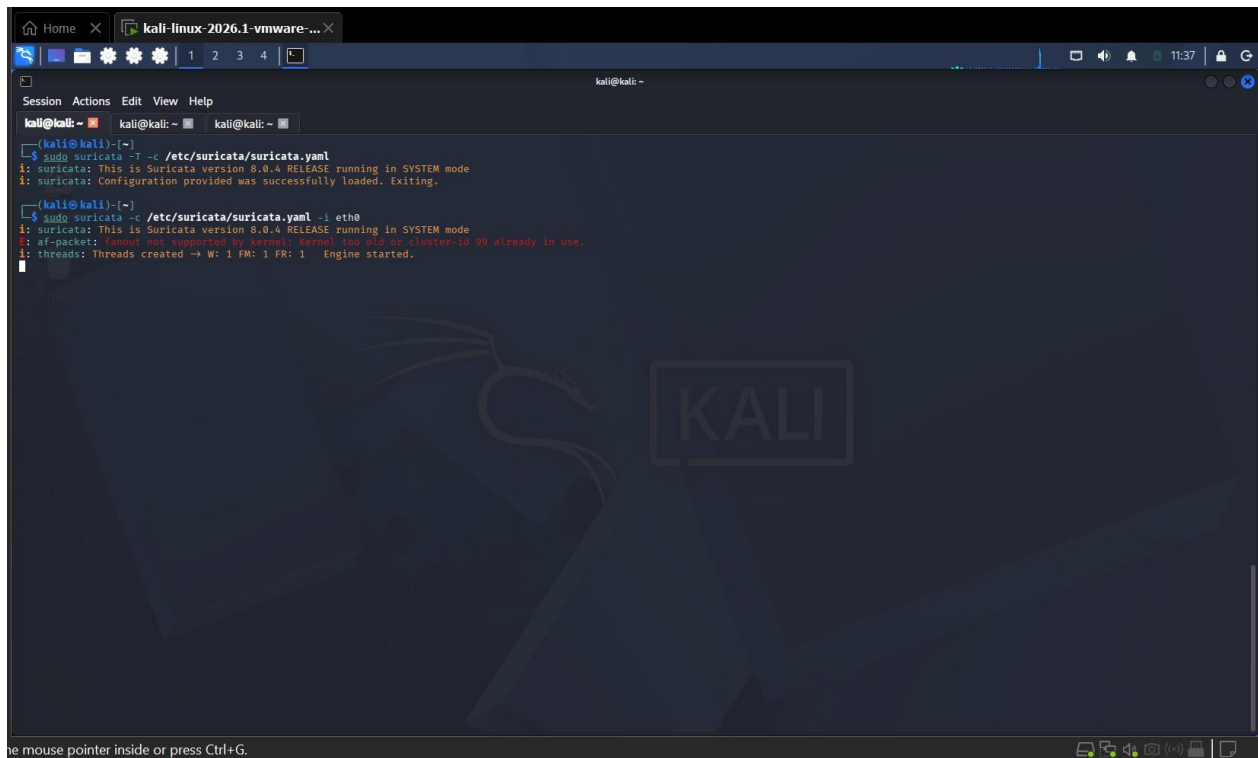
7. Suricata Configuration and Setup

Suricata IDS was configured on Kali Linux using the `suricata.yaml` configuration file. The configuration was verified before running the IDS to ensure that all rules, threshold settings, and logging mechanisms were properly loaded.

After successful validation, Suricata was started on the network interface for live traffic monitoring and packet inspection.

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0
```

This enabled real-time detection of suspicious network activities and alert generation.

A screenshot of a Kali Linux terminal window. The window title is 'kali-linux-2026.1-vmware-...'. The terminal shows the following commands and output:

```
(kali@kali)-[~]
└─$ sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
i: suricata: Configuration provided was successfully loaded. Exiting.

(kali@kali)-[~]
└─$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
i: af-packet: fanout not supported by kernel: kernel too old or cluster-id 99 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
```

The background of the terminal window features a large, faint watermark of a cat's head and the word 'KALI'.

8. Custom Rule Creation

Custom detection rules were created in Suricata to identify different types of network activities such as ICMP traffic, HTTP requests, SYN scans, SSH connections, DNS queries, FTP attempts, MySQL access, and Telnet traffic.

The rules were added inside the `local.rules` file:

```
/var/lib/suricata/rules/local.rules
```

Example rules used in this project:

```
alert icmp any any-> any any (msg:"[CA_TEST] ICMP Ping Detected"; sid:1000001; rev:1;)

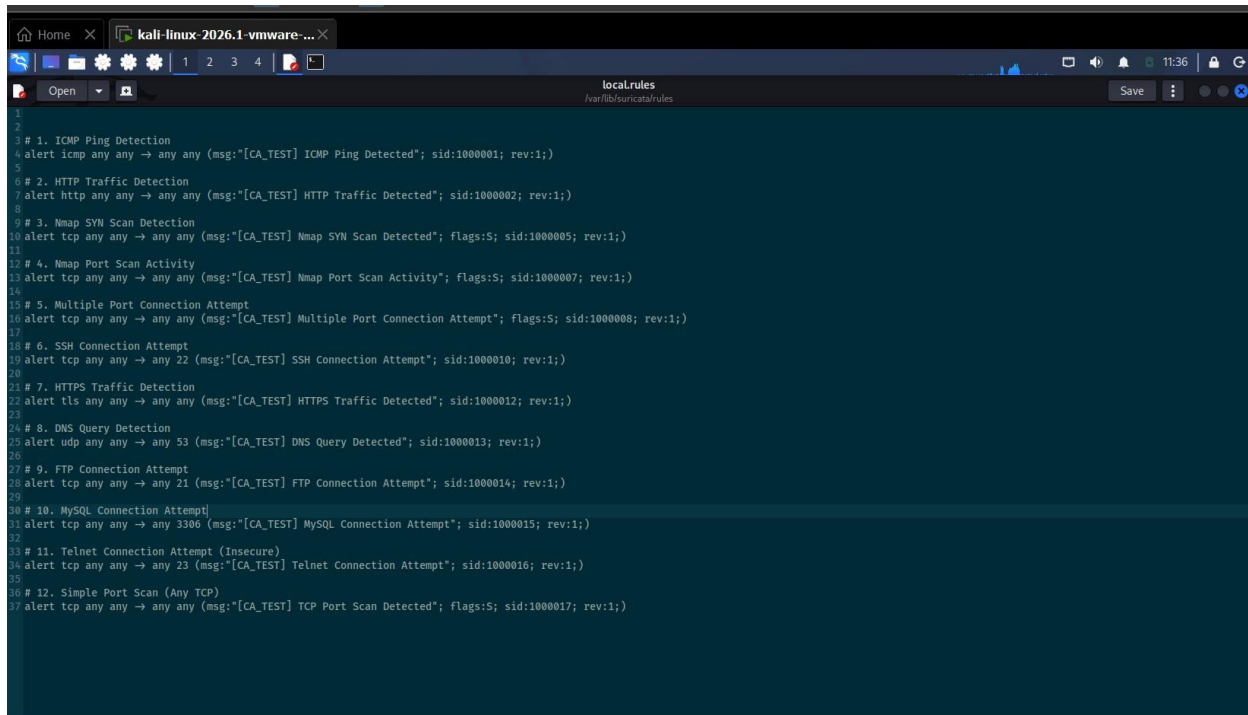
alert http any any-> any any (msg:"[CA TEST] HTTP Traffic Det
```



```
ected"; sid:1000002; rev:1;)
```

```
alert tcp any any-> any any (msg:"[CA_TEST] Nmap SYN Scan Detected"; flags:S; sid:1000005; rev:1;)
```

These rules helped detect and generate alerts for various testing activities performed during the project.

A screenshot of a text editor window titled 'local.rules' showing a list of 12 Suricata rules. The rules are numbered 1 through 12 and cover various network activities like ICMP Ping, HTTP Traffic, Nmap SYN Scan, Nmap Port Scan, Multiple Port Connection Attempt, SSH Connection Attempt, HTTPS Traffic, DNS Query, FTP Connection Attempt, MySQL Connection Attempt, Telnet Connection Attempt, and Simple Port Scan. Each rule includes a comment, an alert action, and specific parameters like message, flags, and sid/rev values.

```
1
2
3 # 1. ICMP Ping Detection
4 alert icmp any any -> any any (msg:"[CA_TEST] ICMP Ping Detected"; sid:1000001; rev:1;)
5
6 # 2. HTTP Traffic Detection
7 alert http any any -> any any (msg:"[CA_TEST] HTTP Traffic Detected"; sid:1000002; rev:1;)
8
9 # 3. Nmap SYN Scan Detection
10 alert tcp any any -> any any (msg:"[CA_TEST] Nmap SYN Scan Detected"; flags:S; sid:1000005; rev:1;)
11
12 # 4. Nmap Port Scan Activity
13 alert tcp any any -> any any (msg:"[CA_TEST] Nmap Port Scan Activity"; flags:S; sid:1000007; rev:1;)
14
15 # 5. Multiple Port Connection Attempt
16 alert tcp any any -> any any (msg:"[CA_TEST] Multiple Port Connection Attempt"; flags:S; sid:1000008; rev:1;)
17
18 # 6. SSH Connection Attempt
19 alert tcp any any -> any 22 (msg:"[CA_TEST] SSH Connection Attempt"; sid:1000010; rev:1;)
20
21 # 7. HTTPS Traffic Detection
22 alert tls any any -> any any (msg:"[CA_TEST] HTTPS Traffic Detected"; sid:1000012; rev:1;)
23
24 # 8. DNS Query Detection
25 alert udp any any -> any 53 (msg:"[CA_TEST] DNS Query Detected"; sid:1000013; rev:1;)
26
27 # 9. FTP Connection Attempt
28 alert tcp any any -> any 21 (msg:"[CA_TEST] FTP Connection Attempt"; sid:1000014; rev:1;)
29
30 # 10. MySQL Connection Attempt
31 alert tcp any any -> any 3306 (msg:"[CA_TEST] MySQL Connection Attempt"; sid:1000015; rev:1;)
32
33 # 11. Telnet Connection Attempt (Insecure)
34 alert tcp any any -> any 23 (msg:"[CA_TEST] Telnet Connection Attempt"; sid:1000016; rev:1;)
35
36 # 12. Simple Port Scan (Any TCP)
37 alert tcp any any -> any any (msg:"[CA_TEST] TCP Port Scan Detected"; flags:S; sid:1000017; rev:1;)
```

9. Traffic Generation and Alert Testing

Different types of network traffic were generated using tools like Ping, Curl, and Nmap to test the Suricata IDS rules and observe alert behavior.

Commands used:

```
(kali@kali)~$ ping -c 8 8.8.8.8
curl -I http://example.com
nmap -sS -p 1-200 -T4 192.168.11.152
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=70.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=203 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=147 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=52.7 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=128 time=90.4 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=128 time=75.5 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=128 time=221 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=128 time=243 ms

--- 8.8.8.8 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7012ms
rtt min/avg/max/mdev = 52.704/137.778/243.045/70.826 ms
HTTP/1.1 200 OK
Date: Thu, 14 May 2026 14:43:09 GMT
Content-Type: text/html
Connection: keep-alive
Server: cloudflare
Last-Modified: Thu, 14 May 2026 05:31:28 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 2735
cf-cache-status: HIT
CF-RAY: 9fbaac725c34288d-NAG

Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-14 10:43 -0400
Nmap scan report for 192.168.11.152
Host is up.
All 200 scanned ports on 192.168.11.152 are in ignored states.
Not shown: 200 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 21.70 seconds
```

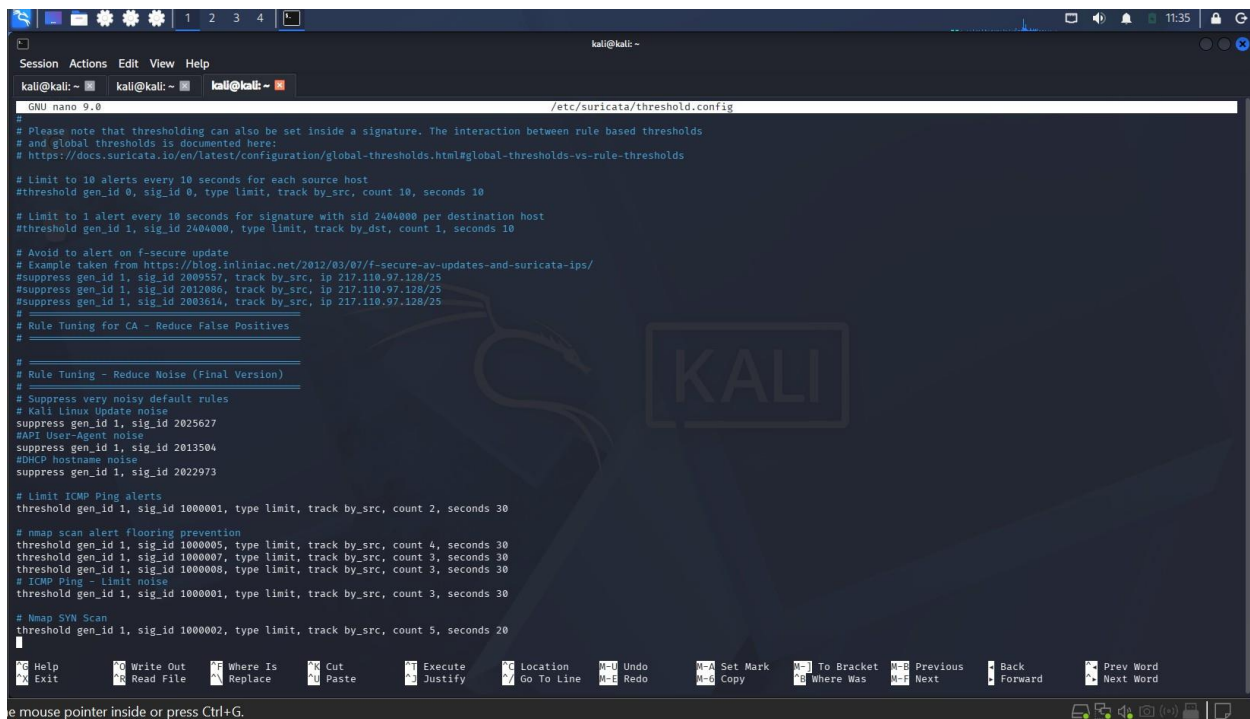
```
ping -c 8.8.8.8
curl -I http://example.com
nmap -sS -p 1-200 -T4 192.168.11.152
```

These commands triggered multiple IDS alerts such as ICMP detection, HTTP traffic alerts, and SYN scan detection.

10. IDS Tuning, Results and Conclusion

Initially, Suricata generated excessive duplicate alerts for repeated network activities, creating alert flooding and increasing false positives.

To solve this issue, thresholding and suppression techniques were implemented using the `threshold.config` file.



```
GNU nano 9.0 /etc/suricata/threshold.config
# Please note that thresholding can also be set inside a signature. The interaction between rule based thresholds
# and global thresholds is documented here:
# https://docs.suricata.io/en/latest/configuration/global-thresholds.html#global-thresholds-vs-rule-thresholds

# Limit to 10 alerts every 10 seconds for each source host
#threshold gen_id 0, sig_id 0, type limit, track by_src, count 10, seconds 10

# Limit to 1 alert every 10 seconds for signature with sid 2404000 per destination host
#threshold gen_id 1, sig_id 2404000, type limit, track by_dst, count 1, seconds 10

# Avoid to alert on f-secure update
# Example taken from https://blog.inliniac.net/2012/02/07/f-secure-av-updates-and-suricata-ips/
#suppress gen_id 1, sig_id 2009557, track by_src, ip 217.110.97.128/25
#suppress gen_id 1, sig_id 2012086, track by_src, ip 217.110.97.128/25
#suppress gen_id 1, sig_id 2003614, track by_src, ip 217.110.97.128/25
# =====
# Rule Tuning for CA - Reduce False Positives
# =====
# Rule Tuning - Reduce Noise (Final Version)
# =====
# Suppress very noisy default rules
# Kali Linux Update noise
#suppress gen_id 1, sig_id 2025627
#API User-Agent noise
#suppress gen_id 1, sig_id 2013504
#DHCP hostname noise
#suppress gen_id 1, sig_id 2022973

# Limit ICMP Ping alerts
#threshold gen_id 1, sig_id 1000001, type limit, track by_src, count 2, seconds 30

# mmap scan alert flooring prevention
#threshold gen_id 1, sig_id 1000005, type limit, track by_src, count 4, seconds 30
#threshold gen_id 1, sig_id 1000007, type limit, track by_src, count 3, seconds 30
#threshold gen_id 1, sig_id 1000008, type limit, track by_src, count 3, seconds 30
# ICMP Ping - Limit noise
#threshold gen_id 1, sig_id 1000001, type limit, track by_src, count 3, seconds 30

# Nmap SYN Scan
#threshold gen_id 1, sig_id 1000002, type limit, track by_src, count 5, seconds 20
```

Example threshold rule:

```
threshold gen_id 1, sig_id 1000001, type limit, track by_src,
count 2, seconds 30
```

- `gen_id 1` → Refers to Suricata's rule generator ID
- `sig_id 1000001` → Targets the ICMP Ping detection rule
- `type limit` → Limits repeated alerts
- `track by_src` → Tracks alerts based on source IP address
- `count 2` → Allows only 2 alerts
- `seconds 30` → Within 30 seconds

Before Tuning:

After tuning:

- Duplicate alerts were reduced
- False positives decreased
- Alert monitoring became easier
- IDS efficiency improved

The project successfully demonstrated how IDS tuning techniques can improve detection accuracy and reduce unnecessary alerts in Suricata IDS environments.

References

Researches - Google Drive

 https://drive.google.com/drive/folders/1AZtVty4HjZshDak3N3UQQm7IN6cU5Q04?usp=drive_link

Georgios P. Spathoulas, Sokratis K. Katsikas, *“Reducing False Positives in Intrusion Detection Systems,”*

Computers & Security, Vol. 29, Issue 1, 2010, pp. 35 44.

Lingala Thirupathi et al., *“Advancements in Intrusion Detection Systems: Exploring Concepts, Innovations, Metrics, Benchmarks, Trends, Directions, Applications and Challenges,”*

International Journal of Progressive Research in Engineering Management and Science IJPREMS , Vol. 4, Issue 12, 2024.

Ali Mustafa, Fouad Trad, Ali Chehab, *“Leveraging Large Language Models for Reducing False Positives and Prioritizing Alerts in Intrusion Detection Systems,”*

American University of Beirut Research Paper, 2024.

El Mostapha Chakir et al., *“False Positives Reduction in Intrusion Detection Systems Using Alert Correlation and Data Mining Techniques,”*

International Journal of Advanced Research in Computer Science and Software Engineering, Vol. 5, Issue 4, 2015.